DTS103TC
Data Structure and Algorithms

Lab 2: Sorting and Searching

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcomes

- Quicksort

- Sequential Search
- Binary Search

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Quicksort

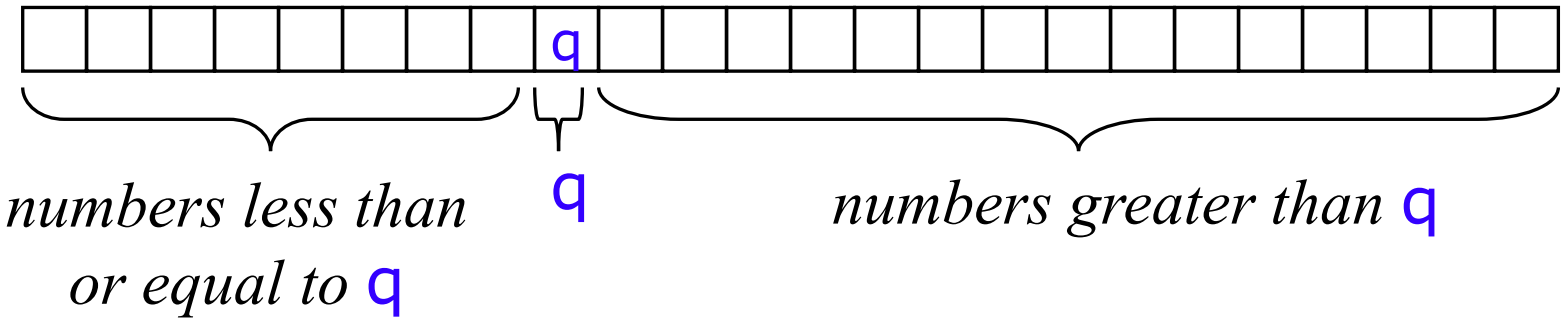
Quicksort

- Divide-and-conquer algorithm
- Sorts "in place" (like insertion sort, but not like merge sort).

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Quicksort

- Pick some number q from the array
- Move all numbers less than (or equal to) q to the beginning of the array
- Move all number greater than q to the end of the array
- Quicksort the numbers less than or equal to q
- Quicksort the numbers greater than or equal to q



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Quicksort pseudo code

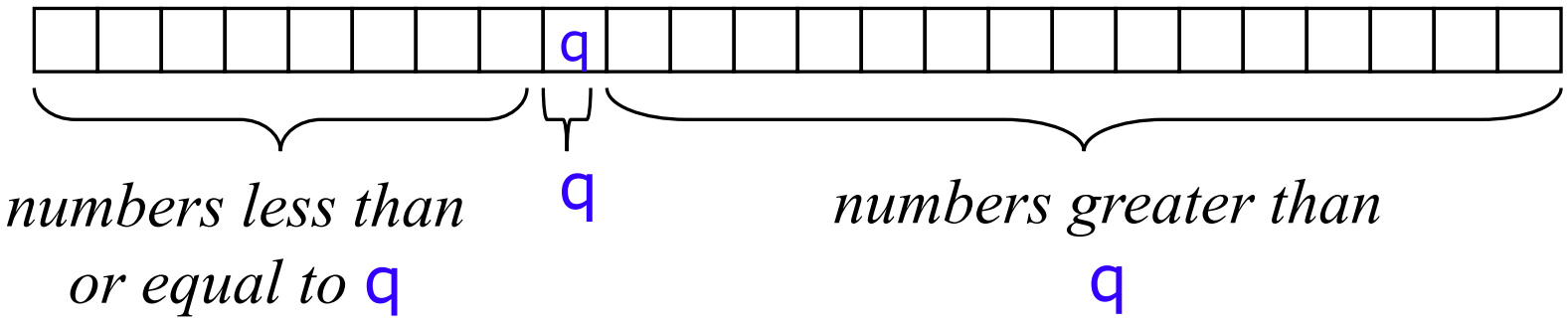
```
Quciksort(A,p,r)
if p<r
        // partition an array such that:
        // all A[p,…,q-1] are less than or equal to A[q]
        // all A[q+1,…,r] are larger than A[q]
        q = Partition(A,p,r)
        Quicksort(A,p,q-1)
        Quicksort(A,q+1,r)
```

Initial call: Quicksort(A,1,n)

Xi'an Jiaotong-Liverpool University 西交利物浦大学

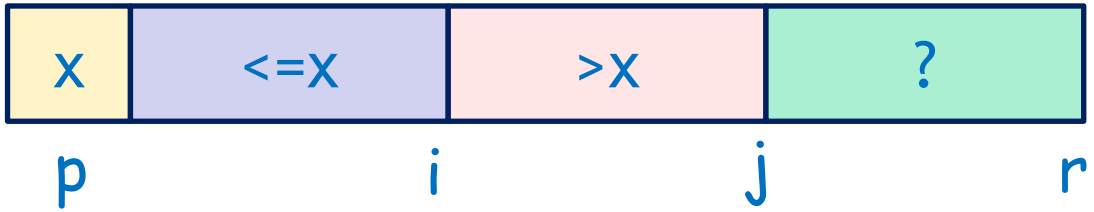
Partitioning

- A key step in the Quicksort algorithm is partitioning the array
  - We choose some (any) number q in the array to use as a pivot
  - We partition the array into three parts:

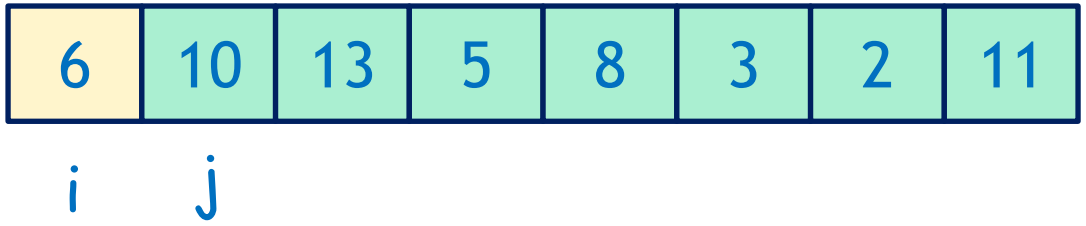


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning

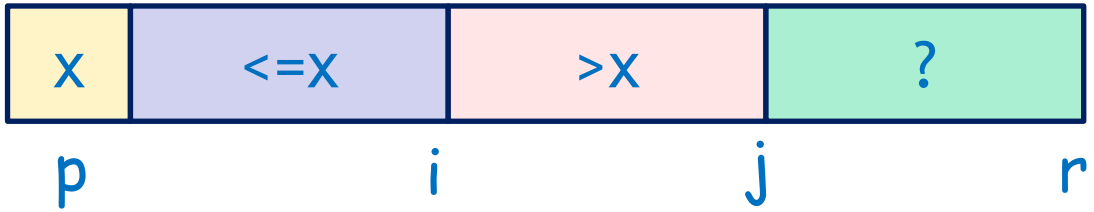



```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```

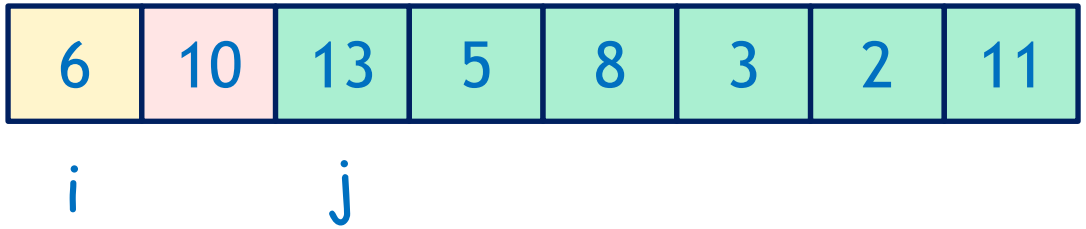


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning

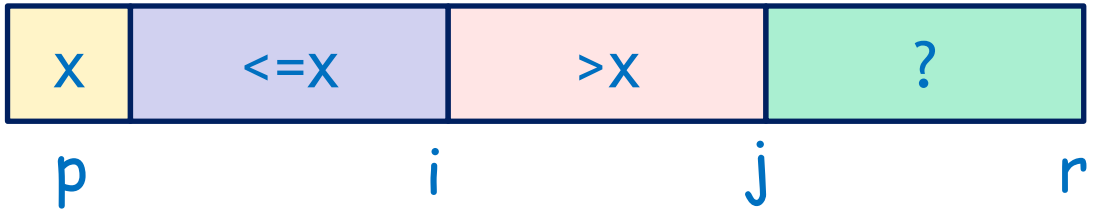



```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```

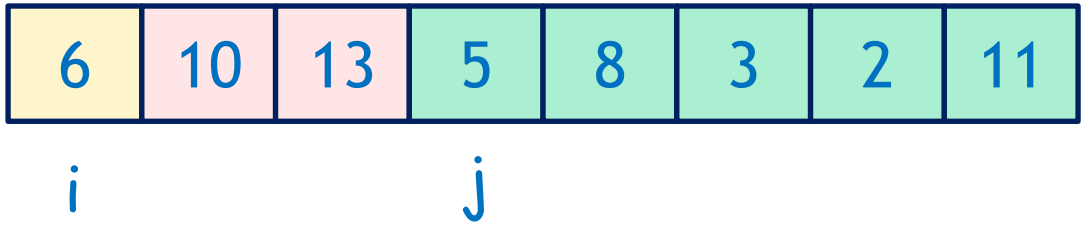


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning



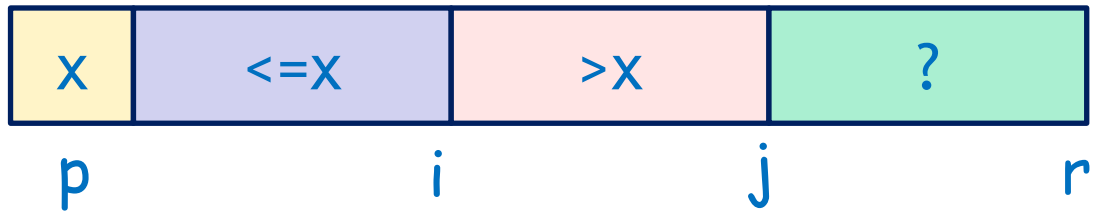

```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```

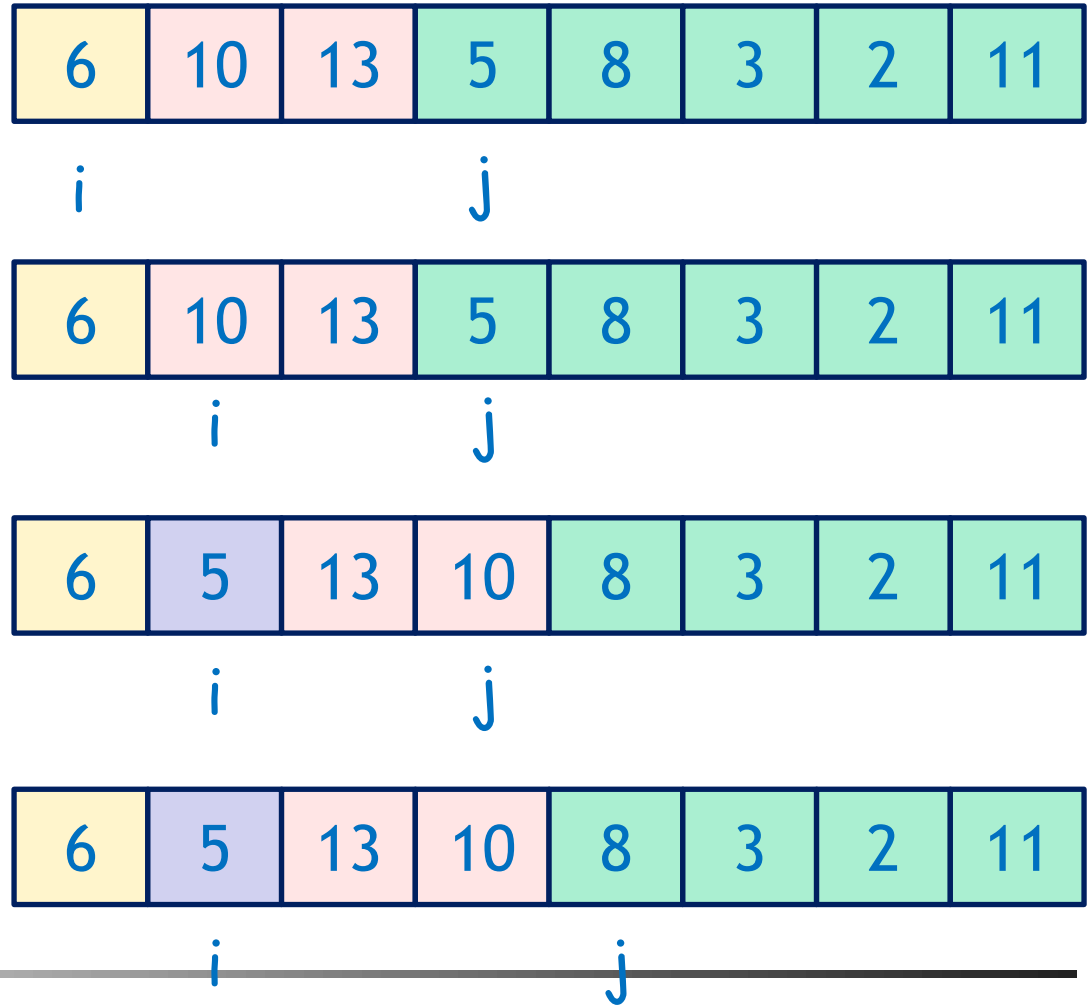


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning

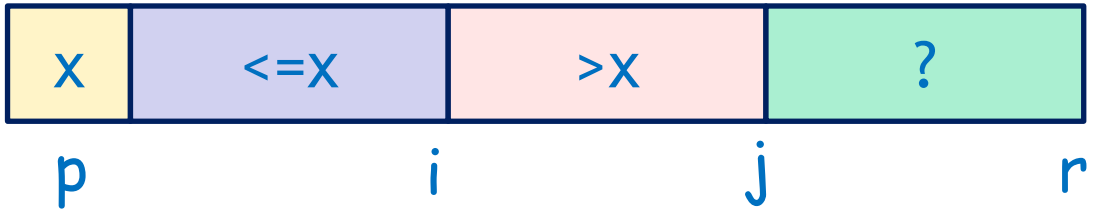
```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```



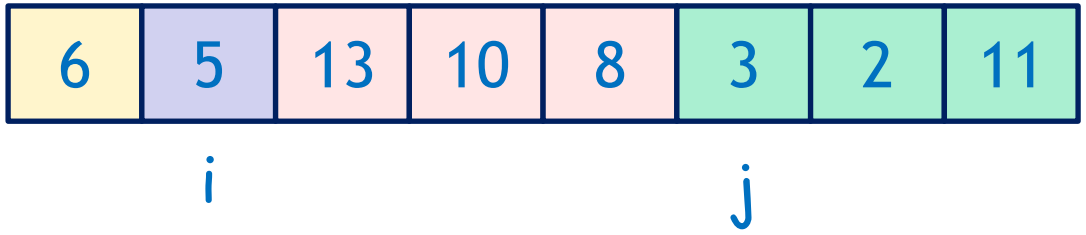


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning



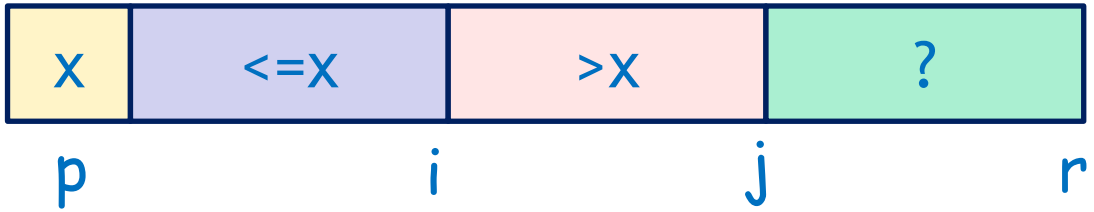

```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```

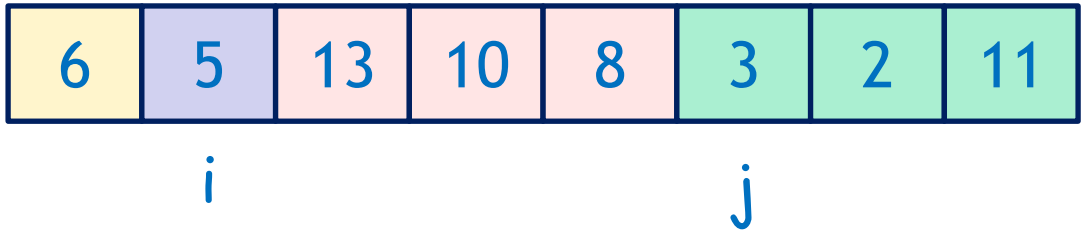


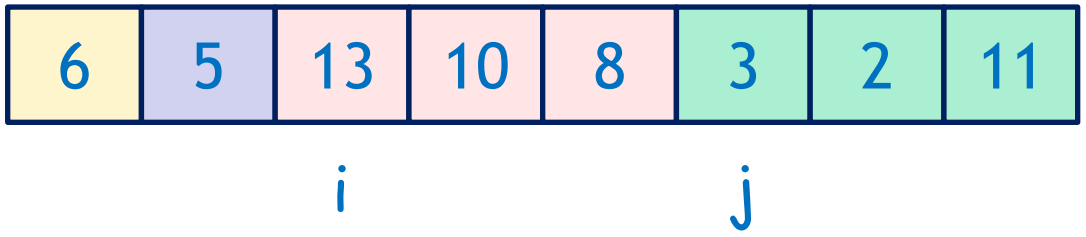
Xi'an Jiaotong-Liverpool University 西交利物浦大学

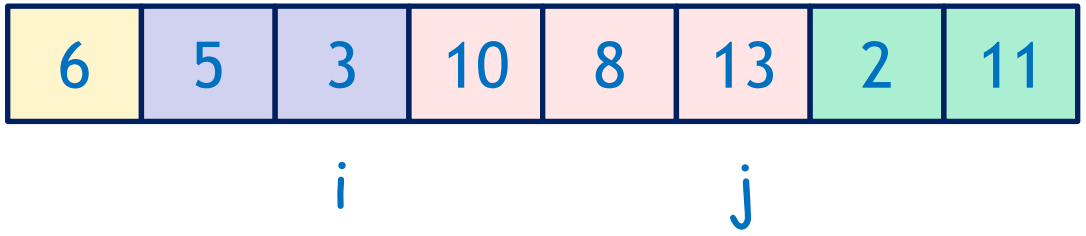
Partitioning

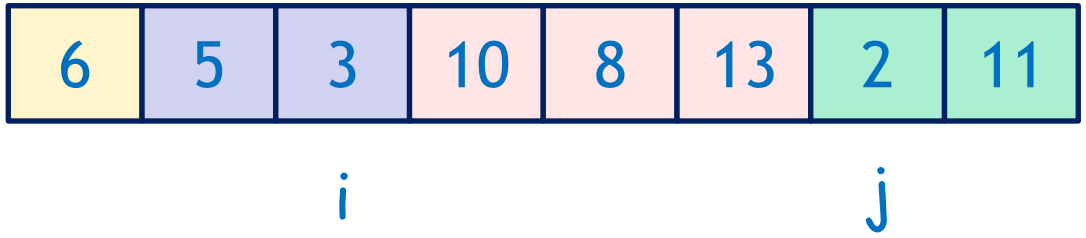
```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```







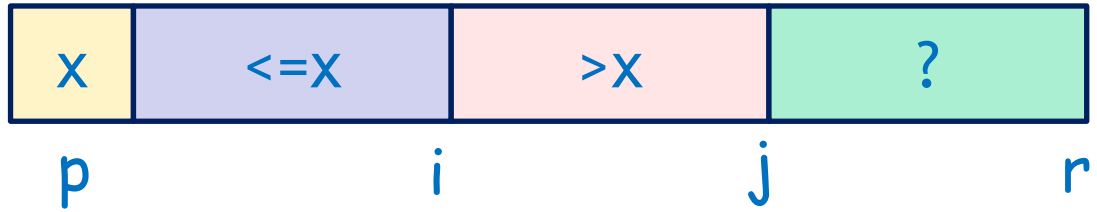


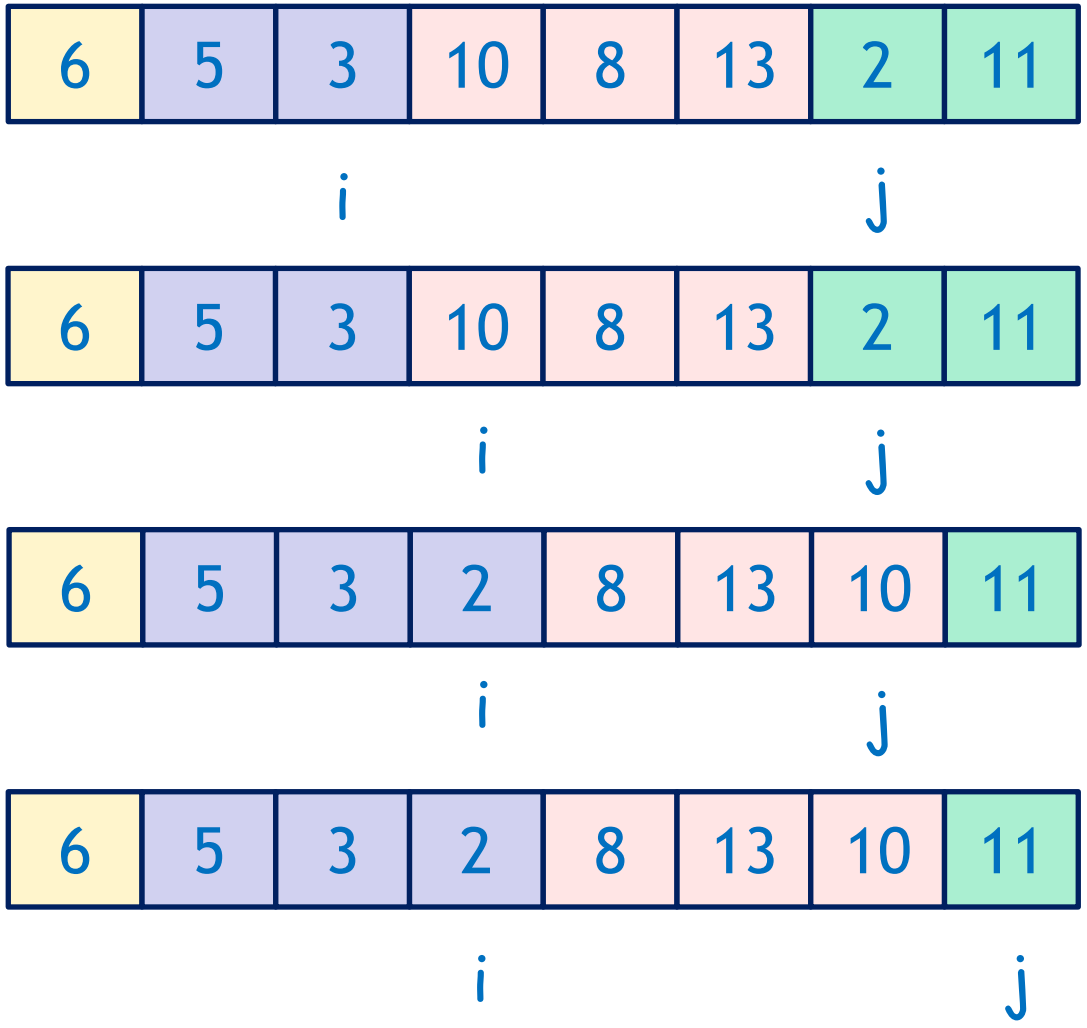


Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning

```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```

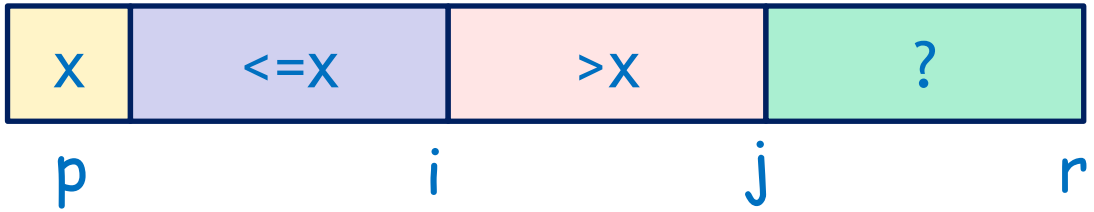


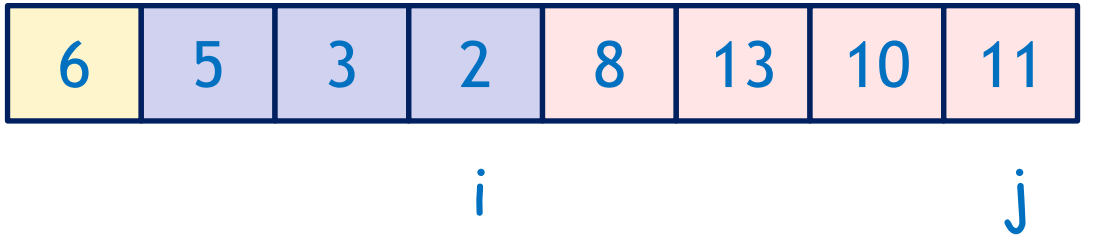


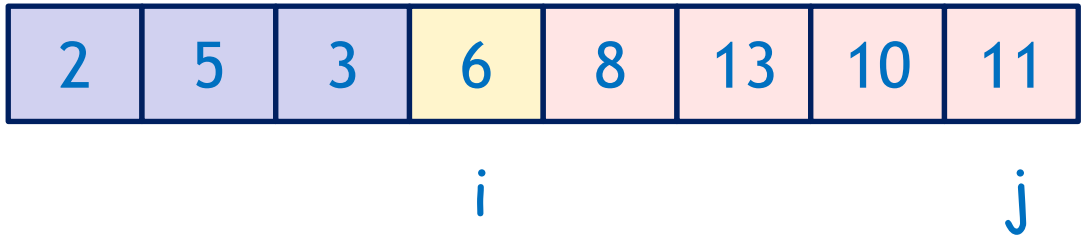
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Partitioning

```
Partition(A,p,r)
x = A[p]
i = p
for j = p+1 to r
    if A[j] <= x
        i=i+1
        exchange A[i] with A[j]
exchange A[p] and A[i]
Return i
```







Xi'an Jiaotong-Liverpool University 西交利物浦大学

Analysis of quicksort

- Best-case
  - Time complexity: $O(n\log n)$
  - Space complexity: $O(\log n)$

- Worst-case
  - Time complexity: $O(n^2)$
  - Space complexty: $O(n)$

- Average-case
  - Time complexity: $O(n\log n)$
  - Space complexity: $O(\log n)$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Searching

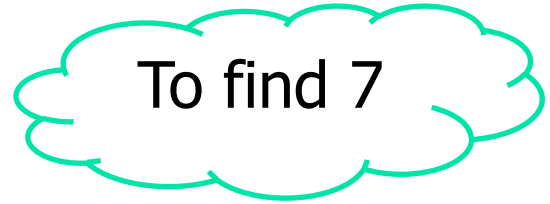
Searching

- Input: n numbers $a_1, a_2, \ldots, a_n$ and a number X
- Output: determine if X is in the sequence or not

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Sequential search

- 12 34 2 9 7 5
  7
- 12 34 2 9 7 5
  7
- 12 34 2 9 7 5
  7
- 12 34 2 9 7 5
  7
- 12 34 2 9 7 5
  7



found!

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Sequential search

- **12** 34 2 9 7 5
  10
- 12 **34** 2 9 7 5
  10
- 12 34 **2** 9 7 5
  10
- 12 34 2 **9** 7 5
  10
- 12 34 2 9 **7** 5
  10
- 12 34 2 9 7 **5**
  10 not found!

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Sequential search

```
i = 1
found = false
while (i<=n && found==false)
{
        if X == a[i] then
                found = true
        else
                i = i+1
}
```

Best case: X is 1st no.
$\Rightarrow$ 1 comparison $\Rightarrow O(1)$

Worst case: X is last OR X is not found $\Rightarrow$ n comparisons $\Rightarrow O(n)$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

How to improve Searching?

- Time complexity of Sequential searching is O(n).

- If a sorted array is given, can we improve the time complexity?

Xi'an Jiaotong-Liverpool University 西交利物浦大学

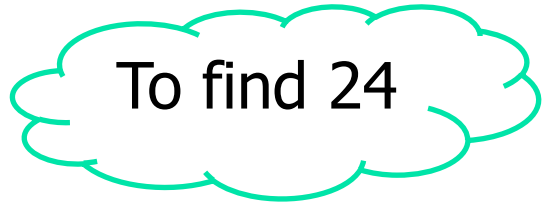
Binary search

- Input: a sequence of n **sorted** numbers $a_1, a_2, \ldots, a_n$ in ascending order and a number X

- Idea of algorithm:
  - compare X with number in the middle
  - then focus on only the first half or the second half (depend on whether X is smaller or greater than the middle number)
  - reduce the amount of numbers to be searched by half

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Binary Search



3 7 11 12 **15** 19 24 33 41 55
24

19 24 **33** 41 55
24

19 24
24

24
24

found!

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Binary Search

To find 30

3 7 11 12 **15** 19 24 33 41 55
30

19 24 **33** 41 55
30

19 24
30

24
30

not found!

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Binary Search – Pseudo Code

```
first = 1, last = n, found = false
while (first <= last && found == false)
{
        mid = ⌊(first+last)/2⌋
        if (X == a[mid])
                found = true
        else
                if (X < a[mid])
                        last = mid-1
                else
                        first = mid+1
}
if (found == true)
        report "Found"
else
        report "Not Found"
```

Best case: X is the number in the middle⇒ 1 comparison ⇒ $O(1)$

Worst case: at most (logn+1) comparisons ⇒ $O(\log n)$

Why?
Every comparison reduces the amount of numbers by at least half
E.g., 16 ⇒ 8 ⇒ 4 ⇒ 2 ⇒ 1

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Recursive Binary Search

```
RecurBinarySearch(A, first, last, X)
begin
   if (first > last) then
        return false
   mid = ⌊(first + last)/2⌋
   if (X == A[mid]) then
        return true
   if (X < A[mid]) then
        return RecurBinarySearch(A, first, mid-1, X)
   else
        return RecurBinarySearch(A, mid+1, last, X)
end
```

invoke by calling
RecurBinarySearch(A, 1, n, X)
return true if X is found,
false otherwise

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Exercises for Lab 2

- Implement Merge Sort (Check Lecture notes)
- Implement Quick Sort
- Implement Sequential Search
- Implement Binary Search
- Test the running time of different algorithms

Xi'an Jiaotong-Liverpool University 西交利物浦大学